# Լաբորատոր աշխատանք 10

**1. Խնդրի Նպատակը**

- Իրականացնել C++ դաս, որն ունակ է ստեղծել դատարկ և տվյալներով օբյեկտներ։
- Ապահովել **դինամիկ զանգվածի չափի անվտանգ փոփոխություն** (հիշողության ճիշտ հատկացմամբ և ազատմամբ)։

**2. Պահանջները**

**Անհատական Պահանջներ (Խնդիր №18 — SavingsPlan)**

- **Տվյալներ:**
  - double* data (ամսական ներդրումներ, $\geq 0$),
  - int n (զանգվածի չափ, $0 \leq n \leq 30$),
  - double p (նպատակային գումար, $p > 0$)։
- **Հաշվարկներ (չեն փոփոխում օբյեկտը):**
  1. Ընդհանուր խնայողություն ($\sum data[i]$)։
  2. Առաջին ամսվա ինդեքսը, երբ կուտակային գումարը հասնում է $p$-ին (եթե չի հասնում՝ $-1$)։
  3. Միջին ներդրումից փոքր ներդրումների քանակը։

**Ընդհանուր Պահանջներ**

- **Կոնստրուկտորներ:**
  - Լռելյայն կոնստրուկտոր (n=0, data=nullptr, p=1)։
  - Պարամետրերով կոնստրուկտոր և դեստրուկտոր։
- **Մեթոդներ:** append(value), removeAt(index), print()։
- **Պատճենման սահմանափակում:** Օբյեկտների պատճենում և վերագրում չկիրառել (delete)։
- **Վավերացում:** Սխալ մուտքային տվյալների դեպքում վերադարձնել false և պահպանել նախկին տվյալները։
- **Ճշտություն:** Իրական թվերի համեմատությունը կատարել $10^{-9}$ճշտությամբ։
- **Դատարկ վիճակ:** Եթե տվյալները բավարար չեն (դատարկ օբյեկտ), արտածել համապատասխան հաղորդագրություն («Տվյալները բավարար չեն»)։

**3. Տեսական Մաս**

- **Լռելյայն Կոնստրուկտոր (Default Constructor):**

Կանչվում է առանց արգումենտների։ Դատարկ օբյեկտի $n = 0$և $data =$ nullptrարժեքները ներկայացնում են վավեր (օրինական) վիճակ։

- **Դինամիկ Հիշողության Կառավարում (Dynamic Memory Management):**

Զանգվածի չափը փոփոխելիս (ավելացնելիս կամ հեռացնելիս) անհրաժեշտ է հետևել 3 քայլին.

1. Հատկացնել **նոր** դինամիկ զանգված (new double[new_size]):
2. Տվյալները պատճենել հին զանգվածից նորի մեջ:
3. Ազատել **հին** հիշողությունը (delete[] data) և ցուցիչը վերագրել նոր հասցեին:

```
#include <iostream>
#include <cmath>
#include <iomanip>

class SavingsPlan {
private:
  double* data; // Ամսական ներդրումների դինամիկ զանգված
  int n;      // Տարրերի քանակը (0 <= n <= 30)
  double p;    // Նպատակային գումար (p > 0)

  // Helper: Իրական թվերի համեմատություն 10^-9 ճշտությամբ
  static bool isNearlyEqual(double a, double b) {
    return std::abs(a - b) < 1e-9;
  }

  // Helper: Իրական թվի համեմատություն (a >= b)
  static bool isGreaterOrEqual(double a, double b) {
    return (a > b) || isNearlyEqual(a, b);
  }

  // Helper: Իրական թվի համեմատություն (a < b)
  static bool isLess(double a, double b) {
    return (a < b) && !isNearlyEqual(a, b);
  }

public:
  // 1. Լռելյայն կոնստրուկտոր (n=0, data=nullptr, p=1.0)
```

```
SavingsPlan() : data(nullptr), n(0), p(1.0) {}

// 2. Պարամետրերով կոնստրուկտոր
SavingsPlan(const double* arr, int size, double targetP) : data(nullptr), n(0), p(1.0) {
    if (targetP <= 0 || size < 3 || size > 30) {
        std::cout << "Սխալ պարամետրեր կոնստրուկտորում! Ստեղծվում է դատարկ օբյեկտ:\n";
        return;
    }

    for (int i = 0; i < size; ++i) {
        if (arr[i] < 0) {
            std::cout << "Սխալ! Ներդրումը չի կարող լինել բացասական:\n";
            return;
        }
    }

    p = targetP;
    n = size;
    data = new double[n];
    for (int i = 0; i < n; ++i) {
        data[i] = arr[i];
    }
}

// 3. Դեստրուկտոր
~SavingsPlan() {
    delete[] data;
}

// Պատճենման և վերագրման արգելում (ըստ պահանջի)
SavingsPlan(const SavingsPlan&) = delete;
SavingsPlan& operator=(const SavingsPlan&) = delete;
```

```
// Set/Get մեթոդ p-ի համար
bool setP(double targetP) {
    if (targetP <= 0) return false;
    p = targetP;
    return true;
}

double getP() const { return p; }
int getSize() const { return n; }

// 4. append(value) — Տարրի ավելացում վերջից
bool append(double value) {
    if (value < 0 || n >= 30) {
        return false;
    }

    double* newData = new double[n + 1];
    for (int i = 0; i < n; ++i) {
        newData[i] = data[i];
    }
    newData[n] = value;

    delete[] data;
    data = newData;
    n++;

    return true;
}

// 5. removeAt(index) — Տարրի հեռացում ըստ ինդեքսի
bool removeAt(int index) {
    if (index < 0 || index >= n) {
        return false;
```

```
    }

    if (n - 1 == 0) {
        delete[] data;
        data = nullptr;
        n = 0;
        return true;
    }

    double* newData = new double[n - 1];
    for (int i = 0, j = 0; i < n; ++i) {
        if (i != index) {
            newData[j++] = data[i];
        }
    }

    delete[] data;
    data = newData;
    n--;

    return true;
}

// 6. print() — Օբյեկտի տվյալների արտածում
void print() const {
    std::cout << "-------------------------------------------\n";
    std::cout << "Նպատակային գումար (p): " << p << "\n";
    std::cout << "Տարրերի քանակ (n): " << n << "\n";
    std::cout << "Ներդրումների ցանկ: ";
    if (n == 0 || data == nullptr) {
        std::cout << "[ Դատարկ (n=0, data=nullptr) ]\n";
    } else {
        std::cout << "[ ";
```

```
        for (int i = 0; i < n; ++i) {
            std::cout << data[i] << (i == n - 1 ? "" : ", ");
        }
        std::cout << " ]\n";
    }
    std::cout << "-----------------------------------------\n";
}

// --- ՀԱՇՎԱՐԿՆԵՐ ---

// Հաշվարկ 1: Ընդհանուր խնայողությունը
double getTotalSavings() const {
    if (n == 0) return 0.0;
    double sum = 0.0;
    for (int i = 0; i < n; ++i) {
        sum += data[i];
    }
    return sum;
}
// Հաշվարկ 2: Առաջին ամսվա ինդեքսը, երբ կուտակային գումարը հասնում է p-ին (-1 եթե չի հասնում)
int getTargetReachedIndex() const {
    if (n == 0) return -1;
    double currentSum = 0.0;
    for (int i = 0; i < n; ++i) {
        currentSum += data[i];
        if (isGreaterOrEqual(currentSum, p)) {
            return i;
        }
    }
    return -1;
}

// Հաշվարկ 3: Միջին ներդրումից փոքր ներդրումների քանակը
```

```
  int getBelowAverageCount() const {
    if (n == 0) {
      std::cout << "Տվյալները բավարար չեն\n";
      return 0;
    }

    double sum = getTotalSavings();
    double avg = sum / n;

    int count = 0;
    for (int i = 0; i < n; ++i) {
      if (isLess(data[i], avg)) {
        count++;
      }
    }
    return count;
  }
};
// Հաշվարկների արտածման օժանդակ ֆունկցիա
void displayResults(const SavingsPlan& plan) {
  plan.print();
  if (plan.getSize() == 0) {
    std::cout << "Ընդհանուր խնայողություն: " << plan.getTotalSavings() << "\n";
    std::cout << "Նպատակին հասնելու ինդեքս: " << plan.getTargetReachedIndex() << "\n";
    std::cout << "Միջինից փոքր ներդրումների քանակ: ";
    plan.getBelowAverageCount();
  } else {
    std::cout << "1. Ընդհանուր խնայողություն: " << plan.getTotalSavings() << "\n";
    std::cout << "2. Նպատակին (p=" << plan.getP() << ") հասնելու առաջին ամսվա ինդեքս: "
          << plan.getTargetReachedIndex() << "\n";
    std::cout << "3. Միջինից փոքր ներդրումների քանակ: "
          << plan.getBelowAverageCount() << "\n";
  }
```

```
    std::cout << "\n";
}
int main() {
    // Փորձնական տվյալներ
    double testPayments[] = {10000, 15000, 8000, 20000};
    int testN = 4;
    double testP = 30000;

    std::cout << "=== 1. ԴԱՏԱՐԿ ՕԲՅԵԿՏ (Լռելյայն p=1.0) ===\n";
    SavingsPlan obj1; // Լռելյայն p=1

    std::cout << "\n--- Ավելացնում ենք օրինակի տարրերը append-ով ---\n";
    for (int i = 0; i < testN; ++i) {
        obj1.append(testPayments[i]);
    }
    std::cout << "\n>>> Օբյեկտ 1-ի արդյունքները (append-ից հետո, p=1.0) <<<\n";
    displayResults(obj1);
    std::cout << "=== 2. ԱՌԱՆՁԻՆ ՕԲՅԵԿՏ ՕՐԻՆԱԿԻ p-ՈՎ (p=30000) ===\n";
    SavingsPlan obj2(testPayments, testN, testP);
    std::cout << "\n>>> Օբյեկտ 2-ի արդյունքները (սկզբնական, p=30000) <<<\n";
    displayResults(obj2);
    std::cout << "=== 3. ՕԲՅԵԿՏ 1-ԻՑ ԻՆԴԵՔՍ 1-Ի ՏԱՐՐԻ ՀԵՌԱՑՈՒՄ (removeAt(1)) ===\n";
    if (obj1.removeAt(1)) {
        std::cout << "Ինդեքս 1-ի տարրը (15000) հեռացվել է հաջողությամբ:\n";
    }
    std::cout << "\n>>> Օբյեկտ 1-ի արդյունքները (հեռացումից հետո) <<<\n";
    displayResults(obj1);

    std::cout << "\n>>> Օբյեկտ 2-ի արդյունքները (անփոփոխ) <<<\n";
    displayResults(obj2);

    return 0;
}
```